

What are different Process States in Operating System ?

Process States

When a program runs, it goes through different stages. These stages are called **process states**.

Think of it like a student completing a task in school.

1. New

- The operating system has just found the program and is preparing to make it a process.
- Like a student getting a new assignment.

Example:

You click on Chrome → OS starts creating a process for Chrome.

2. Ready

- The process is loaded into RAM.
- It is ready to run but waiting for the CPU.
- Like a student standing in a queue, waiting for their turn.

Example:

Chrome is in memory, waiting for CPU time.

3. Running

- CPU is now executing the process.
- Instructions are actively being processed.
- Like the student finally getting their turn and doing the assignment.

Example:

CPU is currently running Chrome and loading tabs.

4. Waiting

- The process cannot continue because it needs some external resource (like input/output).
- Like a student stopping work because they are waiting for the teacher's instructions or a book.

Example:

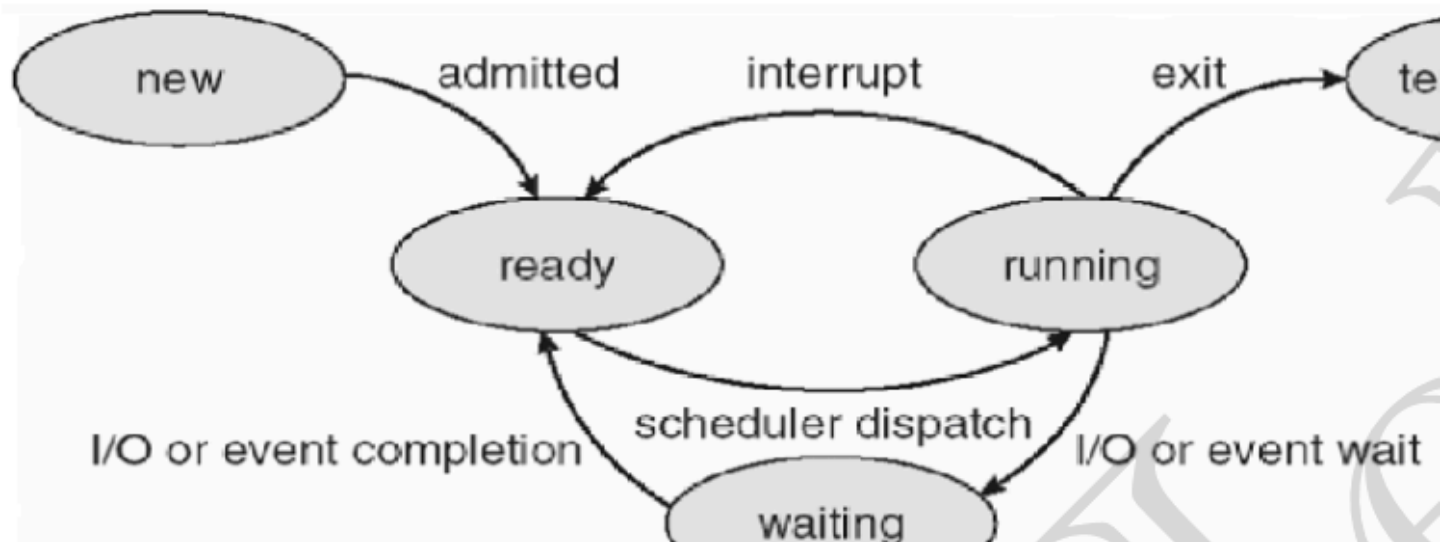
Chrome is waiting for data from the internet → IO wait.

5. Terminated

- Process has completed its task.
- OS will remove its entry from the process table.
- Like the student finishing the assignment and submitting the paper.

Example:

You close Chrome → its process ends.



2. Process Queues

When many processes exist, the OS arranges them in different queues based on their state.

Think of it like queues at different counters in a railway station.

a. Job Queue

Meaning:

This queue stores processes that are **not yet in memory**. They are still in the secondary storage (like hard disk).

Who handles it?

Long-Term Scheduler (LTS)

LTS decides which processes should be moved from the hard disk into RAM.

Analogy:

Students outside the school gate (not entered school yet). Principal picks which students can enter.

Example:

You click 5 apps at once → they are in job queue.

LTS decides which 2–3 apps should be brought into RAM based on available memory.

b. Ready Queue

Meaning:

This holds all processes that are **in RAM** and **ready to run** but waiting for CPU time.

Who handles it?

➡ **Short-Term Scheduler (STS)** (also called CPU scheduler)

STS decides which process gets the CPU next.

Analogy:

Students standing in line inside a classroom, waiting for the teacher to call them.

Example:

Chrome, Spotify, VS Code all are loaded into memory → waiting for CPU.

STS picks one, gives CPU for a short time.

c. Waiting Queue

Meaning:

Contains processes that are **waiting for some event** like I/O, file access, network data, etc.

Analogy:

A student waiting for a book from the library before he can continue his assignment.

Example:

Chrome is waiting for data from the internet → it enters the waiting queue.

3. Degree of Multi-programming

Meaning:

The number of processes present in **main memory (RAM)** at the same time.

More processes in memory → more multi-programming.

Who controls it?

Long-Term Scheduler (LTS)

Because LTS decides how many processes are allowed from job queue → ready queue (i.e., moved to RAM).

Example:

If RAM can hold 5 processes, degree of multi-programming = 5.

If LTS loads 7 processes into RAM (in large systems), degree becomes 7.

4. Dispatcher

The **dispatcher** is a very small but very important part of the OS.

Once the CPU scheduler (STS) selects a process to run → **dispatcher gives the control of CPU to that process.**

Responsibilities of Dispatcher

The dispatcher performs:

1. Context Switching

Saving the current process state and restoring the next process's state.

Example:

Like switching two students on the same computer — one stops, the other continues exactly where he left off.

2. Switching to User Mode

CPU switches from kernel mode to user mode so that the selected process can safely run.

Example:

Teacher allowing a student to use the computer (user mode) after doing internal setup (kernel mode).

3. Jumping to the Right Instruction

After switching, dispatcher jumps to the instruction where the process was paused earlier.

Dispatcher Latency

This is the **time** it takes the dispatcher to complete all this work.

Less latency → faster switching → better performance.

Revision

Job Queue

- Processes in *new state*
- In secondary memory
- Managed by **LTS**

Ready Queue

- Processes in *ready state*
- In RAM
- Managed by **STS**

Waiting Queue

- Processes in *waiting state*
- Waiting for I/O or event

Degree of Multi-programming

- Number of processes in memory
- Controlled by **LTS**

Dispatcher

- Gives CPU to chosen process
- Does **context switch**, **mode switch**, and **PC jump**